세그먼트 트리 (Segment Tree)

문제

크기가 N인 정수 배열 A가 있고, 여기서 다음과 같은 연산을 최대 M번 수행해야 하는 문제가 있습니다.

1. 구간 l, r ($l \le r$)이 주어졌을 때, $A[l] + A[l+1] + \cdots + A[r-1] + A[r]$을 구해서 출력하기
2. i번째 수를 v로 바꾸기 ($A[i] = v$)

1번 연산 $A[l] + A[l+1] + \cdots + A[r-1] + A[r]$을 구하기 위해 소스 1과 같이 모두 더하는 방법이 있습니다.

소스 1

C++ Java Python

```python
ans = 0
for i in range(l, r+1):
    ans += a[i]

```

소스 1의 시간 복잡도는 $O(N)$입니다. 2번 연산 $A[i] = v$는 $O(1)$ 입니다. 연산을 최대 M번 수행해야 하니 연산 하나의 시간 복잡도는 $O(N)$이 됩니다. 총 시간 복잡도는 $O(NM)$입니다.

누적 합

누적 합을 사용하면, 1번 연산의 시간 복잡도를 $O(1)$로 줄일 수 있습니다. 하지만, 2번 연산으로 수가 변경될 때마다 누적 합을 다시 구해야 하기 때문에, 2번 연산의 시간 복잡도는 $O(N)$입니다. 연산 하나의 시간 복잡도는 $O(N)$이니 총 시간 복잡도는 $O(NM)$이 됩니다.

세그먼트 트리

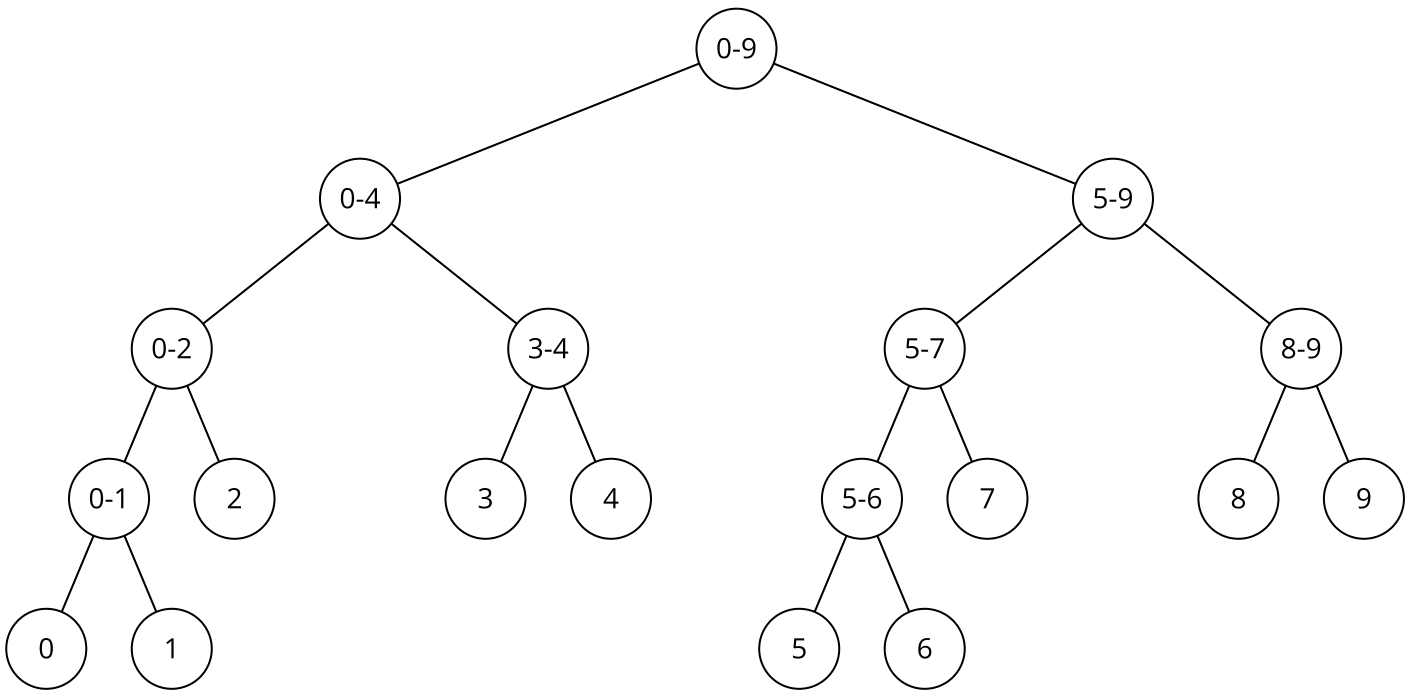
세그먼트 트리를 사용하면 1번 연산과 2번 연산을 $O(\lg N)$에 수행할 수 있습니다.

세그먼트 트리의 리프 노드와 리프 노드가 아닌 노드는 다음과 같은 의미를 가집니다.

- 리프 노드: 배열의 그 수 자체
- 리프 노드가 아닌 노드: 왼쪽 자식과 오른쪽 자식의 합을 저장

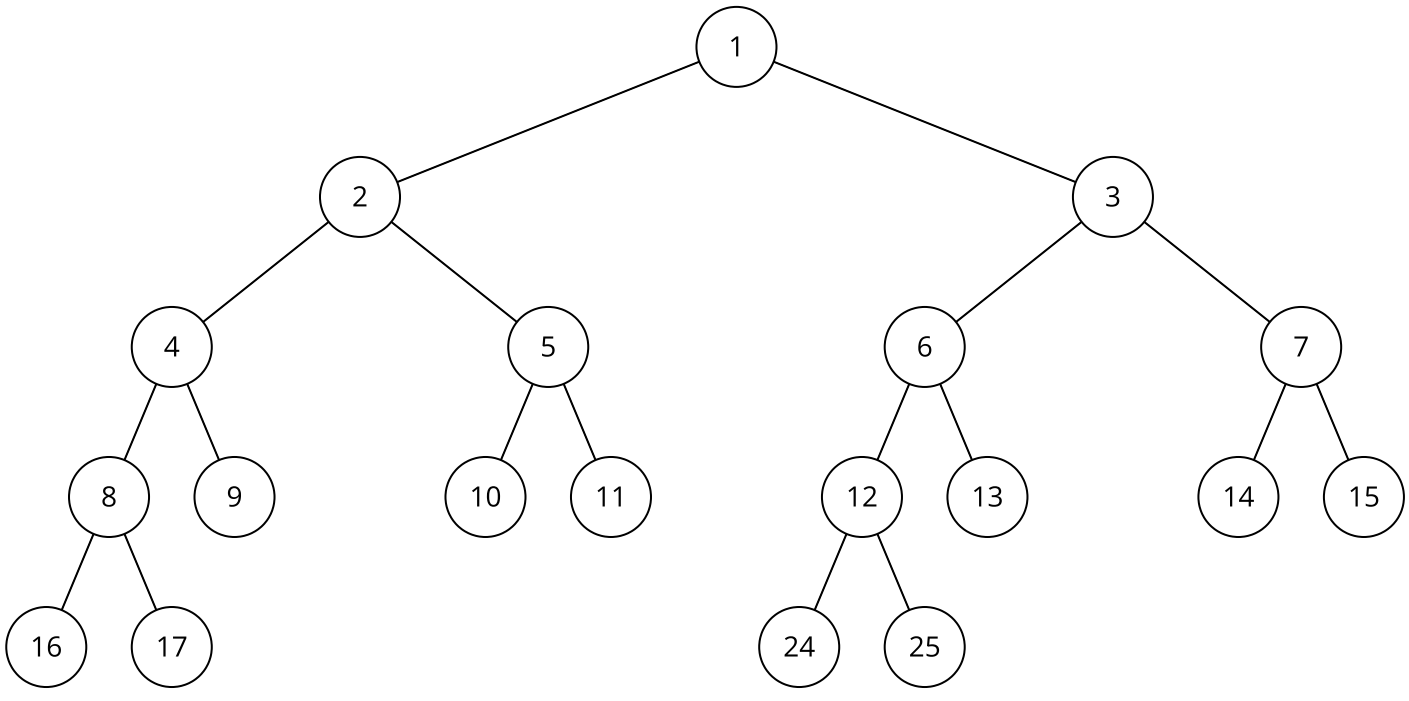
어떤 노드의 번호가 x일때, 왼쪽 자식의 번호는 $2x$, 오른쪽 자식의 번호는 $2x+1$이 됩니다.

다음은 $N =$ 10 인 경우 세그먼트 트리는 다음과 같습니다.



위의 그림은 각 노드가 저장하고 있는 합의 범위를 나타낸 그림입니다. `x-y`는 구간 `[x,y]`를 의미하고, 저장된 값은 $A[x] + \cdots + A[y]$입니다. 리프노드는 `x`로 표현했습니다.

각 노드의 번호를 그림으로 나타내면 다음과 같습니다.



만들기

리프 노드를 제외한 다른 모든 노드는 항상 2개의 자식을 가집니다. 따라서, 세그먼트 트리는 Full Binary Tree의 형태를 가집니다. 만약, N이 2의 제곱꼴인 경우에는 Perfect Binary Tree가 됩니다.

리프 노드가 N개인 Full Binary Tree에는 리프 노드가 아닌 노드가 $N-1$개 있습니다. 따라서, 필요한 노드의 수는 $2N-1$개 있습니다. N이 2의 제곱꼴이 아닌 경우에 높이 $H = \lceil \lg N \rceil$입니다.

세그먼트 트리의 정보를 저장하기 위해서 배열을 사용하겠습니다. 깊이가 가장 깊은 리프 노드와 가장 깊지 않은 리프 노드의 깊이 차이는 1보다 작거나 같습니다. 따라서, 배열을 이용해도 공간을 크게 낭비하지 않습니다. $tree[x]$에 노드 x의 정보를 저장하겠습니다.

높이가 H인 Perfect Binary Tree에 있는 노드의 개수가 배열의 크기가 되고, 이 값은 $2^{H+1}-1$와 같습니다.

소스 2의 `init`은 A를 이용해 세그먼트 트리를 만드는 소스입니다.

소스 2 C++ Java Python

```
# a: 배열 A
# tree: 세그먼트 트리
```

```
# node: 노드 번호
# node에 저장되어 있는 합의 범위가 start - end
def init(a, tree, node, start, end):
    if start == end:
        tree[node] = a[start]
    else:
        init(a, tree, node*2, start, (start+end)//2)
        init(a, tree, node*2+1, (start+end)//2+1, end)
        tree[node] = tree[node*2] + tree[node*2+1]

```

`start == end`인 경우는 리프 노드인 경우입니다. 리프 노드는 배열의 그 수를 저장해야 하기 때문에, `tree[node] = a[start]`가 됩니다.

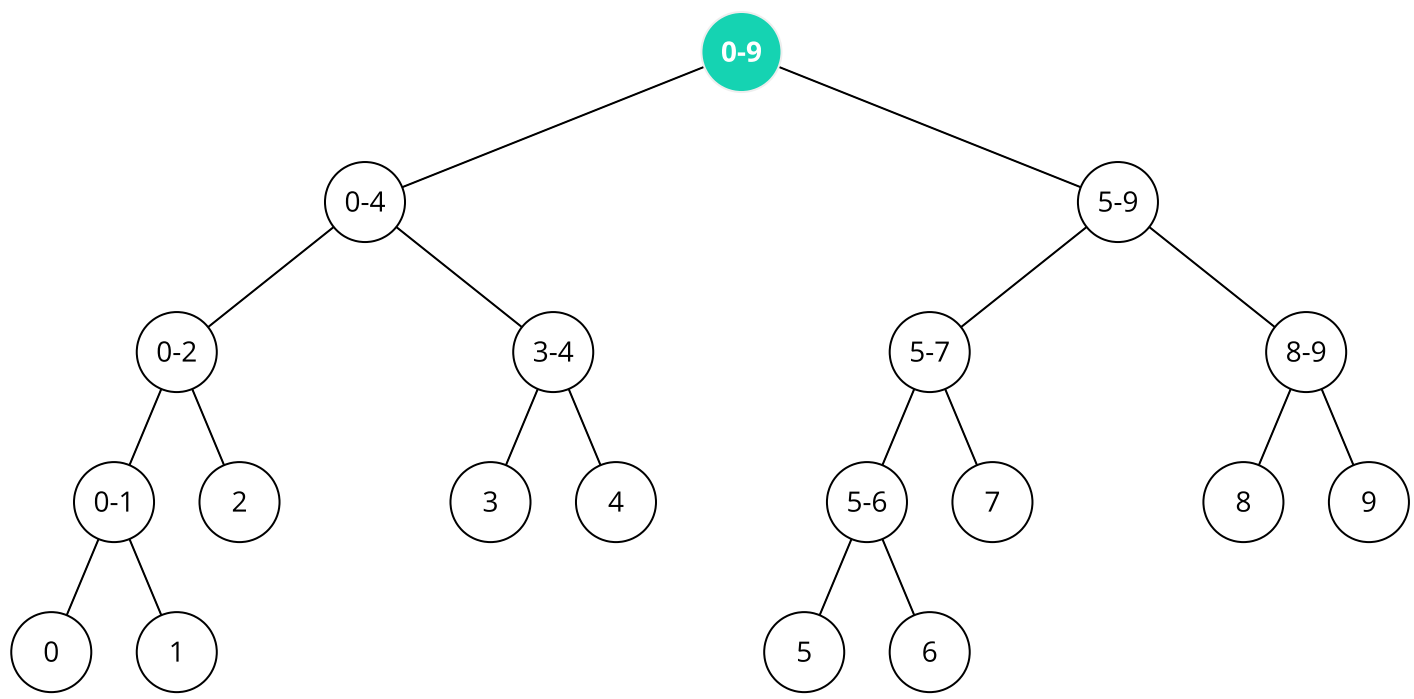
`node`의 왼쪽 자식은 `node*2`이고, 오른쪽 자식은 `node*2+1`입니다. 또, `node`에 저장된 구간이 `[start,end]`라면, 왼쪽 자식은 `[start,(start+end)/2]`, 오른쪽 자식은 `[(start+end)/2+1,end]`가 저장된 구간입니다.

`tree[node]`에 저장될 값을 구하려면 왼쪽 자식에 저장된 값 `tree[node*2]`, 오른쪽 자식에 저장된 값 `tree[node*2+1]`을 먼저 구해야 합니다. 따라서, 재귀 함수를 이용해 각각의 값을 먼저 구했습니다.

구간의 합 구하기

구간 `left`, `right`가 주어졌을 때, 합을 구하려면 트리를 루트부터 순회하면서 각 노드에 저장된 구간의 정보와 `left`, `right`와의 관계를 살펴봐야 합니다.

$N = 10$인 경우 다음과 같은 노드의 정보를 이용해서 합을 구할 수 있습니다.



left = 0, right = 9	**left = 2, right = 4**	**left = 5, right = 8**	**left = 3, right = 9**

`node`에 저장된 구간이 `[start,end]` 이고, 합을 구해야하는 구간이 `[left,right]`라면 다음과 같이 4가지 경우로 나누어질 수 있습니다.

1. `[left,right]`와 `[start,end]`가 겹치지 않는 경우
2. `[left,right]`가 `[start,end]`를 완전히 포함하는 경우
3. `[start,end]`가 `[left,right]`를 완전히 포함하는 경우
4. `[left,right]`와 `[start,end]`가 겹쳐져 있는 경우 (1, 2, 3 제외한 나머지 경우)

1번 경우는 `if (left > end || right < start)`로 나타낼 수 있습니다. `left > end`는 `[start,end]` 뒤에 `[left,right]`가 있는 경우이고, `right < start`는 `[start,end]` 앞에 `[left,right]`가 있는 경우입니다. 이 경우에는 겹치지 않기 때문에, 더 이상 탐색을 이어나갈 필요가 없습니다. 따라서 `0`을 리턴해 탐색을 종료합니다.

2번 경우는 `if (left <= start && end <= right)`로 나타낼 수 있습니다. 이 경우도 더 이상 탐색을 이어나갈 필요가 없습니다. 구해야하는 합의 범위는 `[left,right]`인데, `[start,end]`는 그 범위에 모두 포함되고, 그 `node`의 자식도 모두 포함되기 때문에 더 이상 호출을 하는 것은 비효율적입니다. 따라서, `tree[node]`를 리턴해 탐색을 종료합니다.

3번과 4번의 경우에는 왼쪽 자식과 오른쪽 자식을 루트로 하는 트리에서 다시 탐색을 시작해야 합니다.

소스 3의 `query`는 세그먼트 트리에서 `[left,right]`의 합을 구하는 소스입니다.

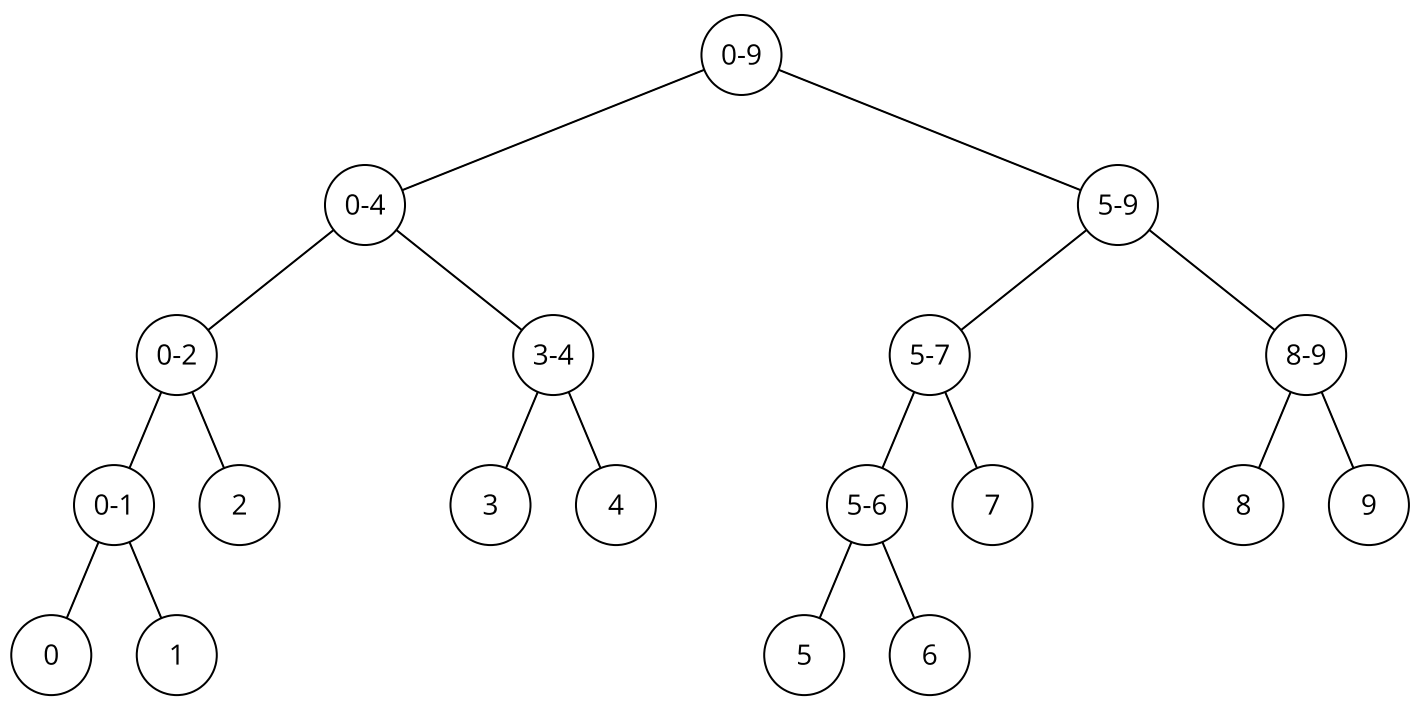
소스 3

C++ Java Python

```python
def query(tree, node, start, end, left, right):
    if left > end or right < start:
        return 0
    if left <= start and end <= right:
        return tree[node]
    lsum = query(tree, node*2, start, (start+end)//2, left, right)
    rsum = query(tree, node*2+1, (start+end)//2+1, end, left, right)
    return lsum + rsum

```

$N =$ 10 , `left =` 3 , `right =` 9 인 경우 합을 구하는 과정 보기



노드 방문

`[left,right]`가 `[start,end]`를 완전히 포함하는 경우

`[left,right]`와 `[start,end]`가 겹치지 않는 경우

그 외

시간 복잡도

트리의 각 레벨에서 방문하는 노드의 개수는 4개를 넘지 않습니다. 트리의 높이 H는 $\lg N$이기 때문에, 합을 구하는 시간 복잡도는 $\lg N$입니다.

첫 번째 레벨에서는 루트 노드 하나만 있고, 루트 노드는 반드시 방문하게 됩니다.

리프 노드가 아닌 노드는 2개의 자식을 갖고, 재귀 호출을 하는 경우 항상 2개의 호출을 하게 됩니다. 어떤 레벨에서 방문한 노드의 개수가 2개 이하인 경우에는 다음 레벨에서 방문한 노드의 개수는 4개 이하입니다.

어떤 레벨에서 방문한 노드의 수가 3개 또는 4개인 경우에 다음 레벨에서 방문한 노드의 수가 4개 이하인지 살펴보면 됩니다.

레벨 l에서 방문한 노드가 3개이고, l, m, r라고 해봅시다. m은 절대로 재귀 호출을 하지 않습니다. 세그먼트 트리의 모든 쿼리는 연속된 구간의 합을 구하게되는데, m은 항상 부모 노드의 구간에 포함되는 노드입니다. 재귀 호출이 일어났다는 것은 그 구간의 일부만 포함되어야 한다는 것을 의미하기 때문입니다. 방문한 노드가 4개인 경우도 가장 왼쪽과 오른쪽에 있는 노드만 재귀 호출을 할 수 있고, 가운데 있는 노드는 재귀 호출을 하지 않습니다.

따라서, 각 레벨에서 최대 4개의 노드만 방문할 수 있습니다.

수 변경하기

`index`번째 수를 `val`로 변경하는 경우, `index`번째를 포함하는 노드에 들어있는 합만 변경해주면 됩니다.

원래 `index`번째 수가 `a[index]`였고, 바뀐 수가 `val`이라면, 합은 `val - a[index]`만큼 변합니다.

수 변경은 다음과 같이 2가지 경우가 있습니다.

1. `[start,end]`에 `index`가 포함되는 경우
2. `[start,end]`에 `index`가 포함되지 않는 경우

1번 경우에만 재귀 호출을 진행하고, 2번 경우는 그 노드의 모든 자식도 `index`번째를 포함하지 않으니 재귀 호출을 중단하면 됩니다.

소스 4의 `update(a, tree, index, val)`은 `index`번째를 `val`로 변경하는 코드이고, 이 함수는 `index`번째를 포함하는 모든 노드의 합에 `diff`를 더해서 수를 변경하는 `update_tree(tree, node, start, end, index, diff)`를 호출하는 소스입니다.

소스 4

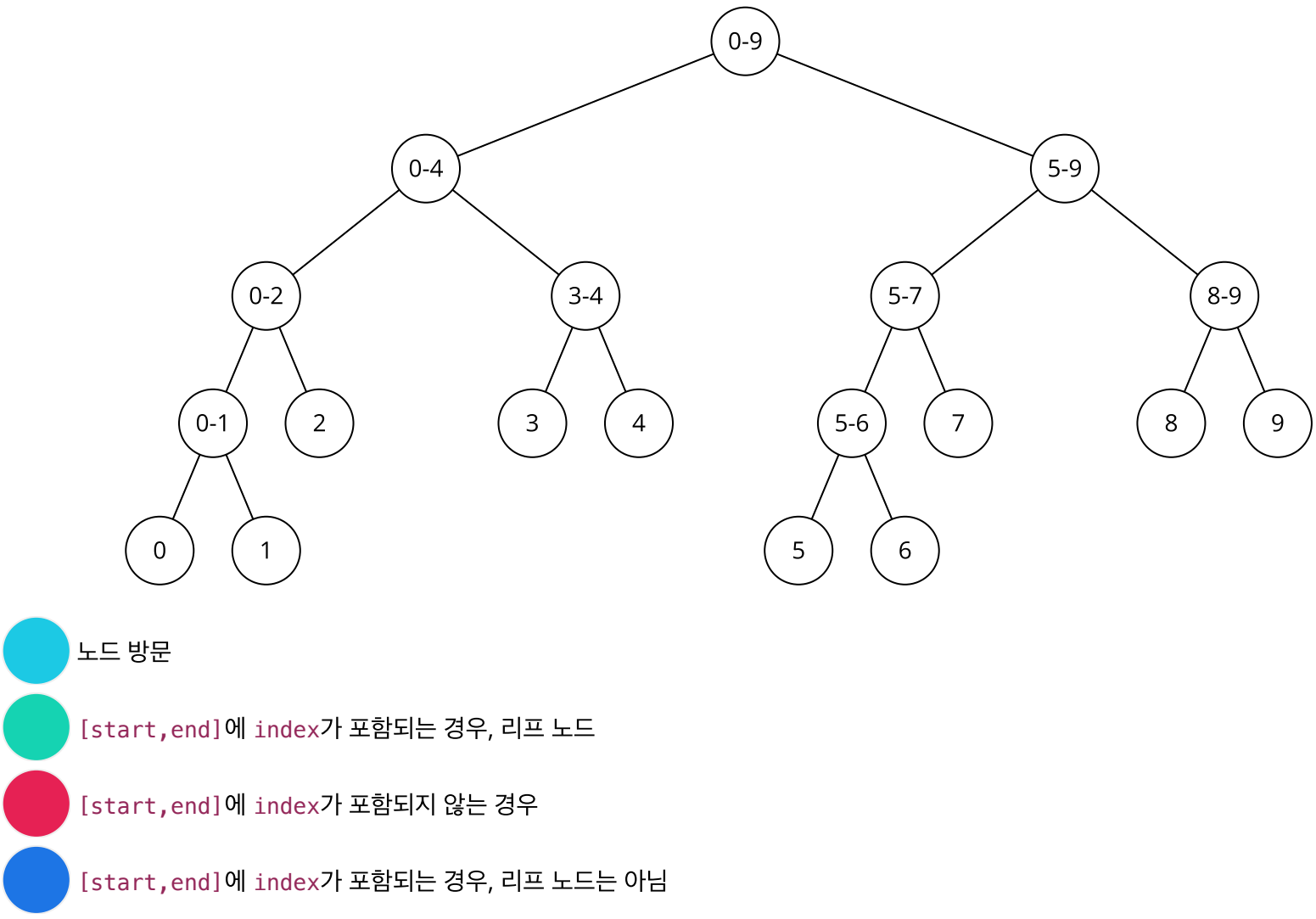
C++ Java Python

```python
def update_tree(tree, node, start, end, index, diff):
    if index < start or index > end:
        return
    tree[node] = tree[node] + diff
    if start != end:
        update_tree(tree, node*2, start, (start+end)//2, index, diff)
        update_tree(tree, node*2+1, (start+end)//2+1, end, index, diff)

def update(a, tree, n, index, val):
    diff = val - a[index]
    a[index] = val
    update_tree(tree, 1, 0, n-1, index, diff)

```

$N =$ 10 , `index =` 3 인 경우 변경하는 과정 보기



수 변경하기 2

위에서 설명한 차이를 이용한 방법 이외에 다른 방법도 있습니다. 먼저, 리프 노드를 찾을때 까지 계속 재귀 호출을 이어나갑니다. 리프 노드를 찾으면 그 노드의 합을 변경해줍니다. 이후 리턴될때마다 각 노드의 합을 자식에 저장된 합을 이용해 다시 구하는 방법도 있습니다.

소스 5의 `update(a, tree, node, start, end, index, val)`은 이 방식을 구현한 소스입니다.

소스 5

C++ Java Python

```python
def update(a, tree, node, start, end, index, val):
    if index < start or index > end:
        return
    if start == end:
        a[index] = val
        tree[node] = val
        return
    update(a, tree, node*2, start, (start+end)//2, index, val)
    update(a, tree, node*2+1, (start+end)//2+1, end, index, val)
    tree[node] = tree[node*2] + tree[node*2+1]

```

시간 복잡도

트리의 각 레벨에서 방문하는 노드의 개수는 2개를 넘지 않습니다. 트리의 높이 H는 $\lg N$이기 때문에, 시간 복잡도는 $\lg N$입니다.

2042 - 구간 합 구하기

이 글의 가장 처음에 설명한 문제와 같은 문제입니다. 소스 6으로 해결할 수 있습니다.

소스 6

C++ Java Python

```python
import sys
import math
input = sys.stdin.readline
def init(a, tree, node, start, end):
    if start == end:
        tree[node] = a[start]
    else:
        init(a, tree, node*2, start, (start+end)//2)
        init(a, tree, node*2+1, (start+end)//2+1, end)
        tree[node] = tree[node*2] + tree[node*2+1]

def update(a, tree, node, start, end, index, val):
    if index < start or index > end:
        return
    if start == end:
        a[index] = val
        tree[node] = val
        return
    update(a, tree, node*2, start, (start+end)//2, index, val)
    update(a, tree, node*2+1, (start+end)//2+1, end, index, val)
    tree[node] = tree[node*2] + tree[node*2+1]

def query(tree, node, start, end, left, right):
    if left > end or right < start:
        return 0
    if left <= start and end <= right:
        return tree[node]
    lsum = query(tree, node*2, start, (start+end)//2, left, right)
    rsum = query(tree, node*2+1, (start+end)//2+1, end, left, right)
    return lsum + rsum

n, m, k = map(int, input().split())
a = [int(input()) for _ in range(n)]
h = math.ceil(math.log2(n))
tree_size = 1 << (h+1)
tree = [0] * tree_size
m += k
init(a, tree, 1, 0, n-1)
for _ in range(m):
    what, t1, t2 = map(int, input().split())
    if what == 1:
        index, val = t1, t2
        update(a, tree, 1, 0, n-1, index-1, val)
    else:
        left, right = t1, t2
        print(query(tree, 1, 0, n-1, left-1, right-1))

```

최솟값

세그먼트 트리를 이용해서 구간의 최솟값도 구할 수 있습니다. 최솟값 이외에도 최댓값, 곱, XOR 연산 등도 구할 수 있습니다.

최솟값을 구하는 경우에는 `+`로 표현한 모든 부분을 `min`으로 바꿔서 구할 수 있습니다.

14438 - 수열과 쿼리 17

구간의 최솟값을 구하는 문제입니다. 소스 7로 해결할 수 있습니다.

소스 7

C++ Java Python

```python
import sys
import math
input = sys.stdin.readline
def init(a, tree, node, start, end):
    if start == end:
        tree[node] = a[start]
    else:
        init(a, tree, node*2, start, (start+end)//2)
        init(a, tree, node*2+1, (start+end)//2+1, end)
        tree[node] = min(tree[node*2], tree[node*2+1])

def update(a, tree, node, start, end, index, val):
    if index < start or index > end:
        return
    if start == end:
        a[index] = val
        tree[node] = val
        return
    update(a, tree, node*2, start, (start+end)//2, index, val)
    update(a, tree, node*2+1, (start+end)//2+1, end, index, val)
    tree[node] = min(tree[node*2], tree[node*2+1])

def query(tree, node, start, end, left, right):
    if left > end or right < start:
        return -1
    if left <= start and end <= right:
        return tree[node]
    lmin = query(tree, node*2, start, (start+end)//2, left, right)
    rmin = query(tree, node*2+1, (start+end)//2+1, end, left, right)
    if lmin == -1:
        return rmin
    elif rmin == -1:
        return lmin
    else:
        return min(lmin, rmin)

n = int(input())
a = list(map(int, input().split()))
h = math.ceil(math.log2(n))
```

```
tree_size = 1 << (h+1)
tree = [0] * tree_size
init(a, tree, 1, 0, n-1)
m = int(input())
for _ in range(m):
    what, t1, t2 = map(int, input().split())
    if what == 1:
        index, val = t1, t2
        update(a, tree, 1, 0, n-1, index-1, val)
    else:
        left, right = t1, t2
        print(query(tree, 1, 0, n-1, left-1, right-1))

```

수를 변경하는 부분이 구간의 합을 구하는 부분과는 조금 차이가 있습니다. 구하고 싶은 구간이 노드가 저장하고 있는 구간과 겹치지 않는 경우 합은 0을 리턴했습니다. 그 이유는 0을 더해도 합에는 변화가 없기 때문입니다. 최솟값에 영향을 미치는 값을 리턴하면 최솟값을 잘못 구할 수 있습니다.

이 문제의 조건을 보면 모든 정수는 1 이상, $1,000,000,000$ 이하라는 조건이 있습니다. -1은 절대로 최솟값으로 나올 수 없는 값이고, 이 값이 리턴된 경우 이 구간에서 최솟값을 구하지 말라는 의미로 사용했습니다. 따라서 -1이 리턴된 경우 다른 한 쪽에서만 최솟값을 구해 리턴하고 있습니다.

어떤 노드 v의 자식이 모두 -1을 리턴하는 경우는 없습니다. 이것은 노드 v의 구간이 구하려고 하는 구간에 포함되지 않는다는 것을 의미하고, 이미 v의 호출에서 -1을 리턴해 호출을 하지 않았을 것이기 때문입니다.

연습 문제

- 2357 - 최솟값과 최댓값
- 11505 - 구간 곱 구하기
- 10868 - 최솟값
- 14428 - 수열과 쿼리 16